

MASTER BUILD GUIDE

The Portfolio Website, Explained Simply

Everything we built — from an empty folder to a live website — written so anyone can understand it, edit it, and explain it in an interview.

Vinith Bonila · Data & AI/ML Engineer

github.com/vinith-bonila

What's inside

- 1 The big picture — what this project actually is
- 2 The toolbox — every tool we used and why (with kid examples)
- 3 How the project is organised — the folder map
- 4 The one file you edit — `portfolio.ts`
- 5 The website, section by section
- 6 The clever features — and how each one works
- 7 How to run it on your own computer
- 8 How to change something and see it
- 9 How to put it on the internet (deploy)
- 10 How to update it later — the 3-command loop
- 11 Interview cheat-sheet — how to explain this project
- 12 Mini-dictionary of every word

The big picture

This project is a **personal portfolio website** — a single web page that shows who you are, the projects you have built, your skills, your experience, and how to contact you. It lives on the internet at its own link, works on phones and laptops, and updates itself whenever you change the code.

IN PLAIN WORDS Think of the website like a house. The tools below are the hammer, bricks, paint, and electricity. This guide explains each one, then shows how we put the house together, and finally how to hand the keys to the internet so everyone can visit.

What makes it special (not a template)

- **One long scrolling page** with a sticky menu at the top that highlights where you are.
- **Dark mode by default**, with a switch to go light — and it remembers your choice.
- **Smooth animations** — text that types itself, numbers that count up, cards that fade in.
- **Clickable project cards** that open a pop-up with full details, screenshots, and live links.
- **A contact form** that lets a visitor pick Gmail, Outlook, or their own mail app.
- **Fast and Google-friendly** — it loads quickly and tells search engines who you are.

The toolbox — every tool and why

A website is built from many small helpers. Here is each one, what it does, and a simple picture to remember it by. If someone in an interview asks *"why did you use X?"*, the one-line answer is in the last column.

React

What it does: The thing that builds the page out of reusable blocks called **components**.

Picture it: LEGO bricks. Instead of one giant wall, you snap together small pieces (a Nav brick, a Hero brick, a Card brick). Change one brick, the rest stay put.

Why we used it: Industry-standard, component-based, huge ecosystem.

Vite

What it does: The **build tool** — it runs the site while you work and bundles it for the internet.

Picture it: A super-fast oven. You change the recipe, and it shows the baked result almost instantly (this is called **hot reload**).

Why we used it: Extremely fast dev server and optimized production builds.

TypeScript

What it does: JavaScript **with a spell-checker**. It catches mistakes before the site runs.

Picture it: A friend reading over your shoulder who says 'you spelled that wrong' before you hand it in.

Why we used it: Fewer bugs; the code documents itself.

Tailwind CSS

What it does: The **styling** system — colours, spacing, fonts — using tiny labels in the HTML.

Picture it: A box of pre-cut, labelled stickers. Instead of mixing paint, you slap on `p-6` (padding) or `text-teal` and it just works.

Why we used it: Fast, consistent styling with no giant CSS files.

Framer Motion

What it does: The **animation** helper — fades, slides, pop-ups, counting numbers.

Picture it: The special-effects team on a movie. You say 'fade in from below', it handles the rest.

Why we used it: Declarative, smooth, respects reduced-motion for accessibility.

Lucide

What it does: The **icon** set — the little pictures (mail, GitHub, arrows).

Picture it: A sticker book of clean, matching icons so nothing looks out of place.

Why we used it: Lightweight, consistent, tree-shakeable icons.

Node & npm

What it does: The **engine** (Node) and the **app store for code** (npm) that installs helpers.

Picture it: npm is like an app store: `npm install` downloads the tools your project lists, the way your phone downloads apps.

Why we used it: Standard way to manage project dependencies.

Git

What it does: The **save system** — it records every version of your code with a history.

Picture it: A video-game with unlimited save slots. You can always go back to an earlier save.

Why we used it: Version control; safe, reversible changes.

GitHub

What it does: The **cloud locker** where your Git saves live online.

Picture it: Google Drive, but for code. Your project has a home on the internet that Vercel can read.

Why we used it: Hosts the repo and triggers auto-deploys.

Vercel

What it does: The **robot** that takes your GitHub code and publishes it live, automatically.

Picture it: A delivery robot: every time you push code to GitHub, it rebuilds the site and puts the new version online in under a minute.

Why we used it: Zero-config hosting for Vite; auto-deploy on every push.

PART 3

How the project is organised

Every file has a job. Here is the map of the important ones. You rarely touch most of them — the one you WILL edit is highlighted.

```

Portfolio_Vinith_Bonila/
├─ index.html           the empty page the site loads into (+ SEO tags)
├─ package.json         the shopping list of tools npm installs
├─ vite.config.ts       settings for the build tool
├─ tailwind.config.js   your colours + fonts live here
├─ vercel.json          tells Vercel how to build the site
├─ public/             files served as-is
│   └─ resume.pdf       <-- your CV (swap this file to update it)
│   └─ og-image.png     the preview card shown when you share the link
│   └─ robots.txt      a note for Google's robots
│   └─ images/          project screenshots
└─ src/                the actual website code
    └─ data/portfolio.ts <== ALL your words live here. EDIT THIS.
    └─ components/      the LEGO bricks (Hero, Projects, Contact...)
    └─ hooks/           small reusable behaviours (theme, scroll-spy)
    └─ App.tsx          snaps all the bricks together in order
    └─ index.css        global colours + the grid/noise background

```

IN PLAIN WORDS If you only remember one line from this whole guide: to change words, edit `src/data/portfolio.ts` . To change the CV, replace `public/resume.pdf` . You do not need to touch anything else.

PART 4

The one file you edit — portfolio.ts

We deliberately put **all the text** of the website into a single file: `src/data/portfolio.ts` . The bricks (components) read from it. This means you change your job title, projects, or phone number in one place, and the whole site updates — without touching any complicated code.

Example: changing your tagline

Find this near the top of the file:

```

export const hero = {
  name: "Vinith Bonila",
  status: "Open to opportunities",
  tagline: "Ships production-grade AI systems end to end.",
  ...
}

```

Change the words inside the quotes, save the file, and the homepage headline updates. That is it. No other file needs to change.

What lives in this file

Section in the file	Controls on the website
hero	Name, rotating job titles, tagline, buttons
projects	Every project card + its pop-up (details, metrics, links, images)
skills	The skills grid, grouped by category
timeline	Experience + education (ONGC, ShutrX, degree)

Section in the file	Controls on the website
certifications	The certificates strip
proofOfWork	The count-up numbers band under the hero
systemProfile	The little 'system profile' panel in the hero
currently / contact	The 'open to' list, email, phone

PART 5

The website, section by section

Each section is one component (one LEGO brick) in `src/components/` . In order down the page:

Nav.tsx

The sticky top menu. It highlights the section you're looking at (called **scroll-spy**), animates the underline, and holds the dark/light switch and the mobile hamburger menu.

Hero.tsx

The first thing visitors see: your big name, the job title that types itself, your tagline, the two buttons, social icons, and the 'system profile' panel on the right.

ProofOfWork.tsx

The band of numbers under the hero (2 live AI systems, 24+ tests, 19 intents...) that count up from zero when they scroll into view.

Projects.tsx + ProjectModal.tsx

The heart of the site. A grid of clickable cards; clicking one opens a pop-up (**modal**) with the full story — overview, screenshots, an architecture diagram, count-up metrics, tech list, and Live + GitHub links.

Skills.tsx

Your skills, grouped into boxes (Programming, AI/ML, Backend, Data...). Each pill lifts a little when you hover it.

Experience.tsx

A vertical timeline for ONGC, ShutrX, SPE, and your degree. The teal line 'draws itself' as you scroll. Certificates sit underneath.

Contact.tsx + ContactForm.tsx

Your big email button with a Copy button, plus the contact form that lets visitors compose in Gmail / Outlook / their own mail app. Footer at the bottom.

PART 6

The clever features — how each works

Dark / light mode that remembers

A switch flips a `dark` label on the page and saves your choice in the browser's memory (**localStorage**). Next visit, it remembers.

Picture it: Like a light switch in a room that stays how you left it when you come back.

Text that types itself (Typewriter)

The job title under your name is typed out letter by letter, then deleted, then the next one types — forever looping through your roles.

Picture it: Like watching someone type a text message, on repeat.

Numbers that count up

Metrics (like 1.00, 19, 24+) animate from 0 to the real number when they appear on screen.

Picture it: Like a scoreboard rolling up to the final score.

Pop-up project details (Modal)

Clicking a card opens a window on top of the page with a blurred background. Press Esc or click outside to close. The keyboard is 'trapped' inside so it stays accessible.

Picture it: Like a photo you tap to enlarge, then tap away to close.

Magnetic buttons

The main buttons gently lean toward your mouse when you hover near them.

Picture it: Like a fridge magnet being pulled toward metal.

Keyboard shortcuts (a hidden power-user touch)

Press **g** then **p** to jump to Projects, **g** then **c** for Contact, or **?** to see the full list. It shows you know your craft.

Picture it: Like keyboard shortcuts in a video game for people who know them.

The contact form's app picker

After a visitor writes a message, they pick where to send it from: Gmail, Outlook, their default mail app, or copy the text. Each opens pre-filled with their message to you.

Picture it: Like 'Open with...' on your phone when you tap a link.

Google name-tag (SEO) + share preview (OG image)

Hidden tags tell Google 'this page is about Vinith Bonila, a Data/AI engineer'. A picture (og-image.png) shows up when the link is shared on LinkedIn or WhatsApp.

Picture it: Like a name badge for robots, and a thumbnail for your link.

Respects 'reduce motion'

If a visitor's device is set to reduce animations (for comfort or accessibility), the site quietly turns them off.

Picture it: Like a considerate host dimming the lights for a guest who asked.

PART 7

How to run it on your own computer

You need **Node.js** installed once (from nodejs.org). Then open a terminal in the project folder and run two commands:

in the project folder

```
npm install      # downloads all the tools (only needed the first time)
npm run dev      # starts the site on your computer
```

It prints a link like `http://localhost:5173`. Open that in your browser. While `npm run dev` is running, any change you save appears instantly (that is Vite's hot reload).

IN PLAIN WORDS `localhost` means 'this computer only' — it is a private preview. Nobody else can see it until you deploy (Part 9).

Other useful commands

Command	What it does
<code>npm run dev</code>	Preview the site locally while you work
<code>npm run build</code>	Make the final, optimised version (into a <code>dist</code> folder)
<code>npm run preview</code>	Preview that final build locally

PART 8

How to change something and see it

The everyday loop is tiny:

- **Open** `src/data/portfolio.ts` (for words) in an editor like VS Code.
- **Edit** the text inside the quotes and save.
- **Look** at the browser — with `npm run dev` running, it already updated.

Common edits

You want to change...	Do this
Any text / job title / project	Edit <code>src/data/portfolio.ts</code>
Your CV/resume	Replace <code>public/resume.pdf</code> (keep the same name)
A project screenshot	Replace the file in <code>public/images/</code> (keep the same name)
The accent colour (teal)	Change <code>colors.accent</code> in <code>tailwind.config.js</code>
Your domain in SEO tags	Find-and-replace <code>your-domain.com</code> in <code>index.html</code> , <code>robots.txt</code> , <code>sitemap.xml</code>

PART 9

How to put it on the internet (deploy)

We used **GitHub + Vercel**. You do this set-up **once**.

One-time setup

- **Save the project to Git** (a first snapshot).
- **Create an empty repository** on `github.com` and push the code up to it.
- **Import that repo into Vercel** — it detects Vite automatically and clicks Deploy.
- In under a minute you get a **live link** like `your-site.vercel.app`.

the one-time push to GitHub

```
git init
git add .
git commit -m "Initial portfolio"
git branch -M main
git remote add origin https://github.com/vinith-bonila/portfolio-website.git
git push -u origin main
```

IN PLAIN WORDS After this, GitHub and Vercel are 'married'. You never re-import. Every future push updates the live site by itself.

PART 10

How to update it later — the 3-command loop

Once it is live, every change follows the exact same three commands:

run these after any edit

```
git add .
git commit -m "describe what you changed"
git push
```

The moment `git push` finishes, **Vercel notices and rebuilds automatically**. Your live site refreshes itself in about 30–60 seconds. No dashboards, no clicking.

- `git add .` — gather up everything you changed.
- `git commit -m "..."` — save a labelled snapshot.
- `git push` — send it to GitHub → Vercel auto-deploys.

IN PLAIN WORDS If a change has a coding mistake, Vercel keeps the last working version live and shows the error in its log — so a bad push can never take your site down.

PART 11

Interview cheat-sheet

Here is how to talk about this project confidently. Start with a 20-second summary, then go deeper only if they ask.

Your 20-second pitch

SAY THIS "I built my portfolio from scratch as a single-page React and TypeScript app, styled with Tailwind and animated with Framer Motion. All the content is in one data file so it's easy to maintain, it's fully responsive with a dark mode, accessible with keyboard support and reduced-motion, and it auto-deploys to Vercel on every Git push."

If they ask 'why these tools?'

- **React** — component reuse; the industry standard, so it shows employable skills.

- **TypeScript** — catches errors early and makes the code self-documenting.
- **Vite** — fast dev feedback and a small, optimised production build.
- **Tailwind** — consistent styling quickly, without a sprawling CSS file.
- **Framer Motion** — smooth animations that also respect accessibility settings.
- **Vercel** — zero-config hosting with automatic deploys straight from GitHub.

Good things to mention (they show maturity)

- **Accessibility:** keyboard-navigable pop-ups with focus-trapping, alt text, and `prefers-reduced-motion` support.
- **Performance:** below-the-fold sections load lazily; images lazy-load; no layout shift.
- **Maintainability:** a single content file (`portfolio.ts`) separates words from code.
- **SEO:** structured data (Schema.org Person), sitemap, robots, and social share image.
- **DevOps:** Git version control and CI-style auto-deploys on every push.

Likely questions and short answers

Question	Short answer
Why a single-page site?	A portfolio is one story; scrolling keeps it fast and simple.
How is it responsive?	Tailwind's mobile-first breakpoints; tested on phone and desktop.
How does dark mode persist?	The choice is saved in the browser's localStorage.
Is there a backend?	No — it's fully static, so it's fast, cheap, and can't 'go down'.
How does the contact form work?	It builds a pre-filled email and lets the visitor pick Gmail, Outlook, or their default app — no server needed.
How do you deploy?	Push to GitHub; Vercel auto-builds and publishes in under a minute.

PART 12

Mini-dictionary

Component — A reusable block of the page (like the Nav or a project Card).

Props / State — Data passed into a component, and data it remembers while running.

Hook — A small reusable behaviour in React (e.g. the theme toggle).

Hot reload — Vite showing your saved change instantly while you work.

Build — Turning the source code into the final files browsers load.

Responsive — The layout adapts to phone, tablet, and desktop screen sizes.

Breakpoint — A screen width where the layout changes (Tailwind's sm / md / lg).

Modal — A pop-up window layered over the page.

Scroll-spy — Highlighting the menu item for the section you're viewing.

localStorage — A small memory in the browser that survives refreshes.

Lazy load — Loading a part of the page only when it's needed, to start faster.

Accessibility (a11y) — Making the site usable with a keyboard and screen readers.

mailto — A special link that opens an email pre-addressed and pre-filled.

SEO — Search Engine Optimisation — helping Google understand and rank the page.

OG image — The preview picture shown when a link is shared on social apps.

Repository (repo) — A project's folder tracked by Git and stored on GitHub.

Commit — One saved snapshot of your code with a message.

Push — Sending your commits up to GitHub.

Deploy — Publishing the site so the public internet can see it.

Static site — A site with no server logic — just files; fast and reliable.

Written as a companion to the Vinith Bonila portfolio project. Keep this file with the project so anyone who edits it later has the full picture.